

Semana 14 - P1

Raíces polinómicas y ajuste de datos

Unidad de repaso aplicado: `np.roots` y `np.polyfit`

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 14, Sesión 1 (Sesión 27)

Dos Problemas Frecuentes

Resolver Polinomios con `np.roots`

Ajustar Datos con `np.polyfit`

Ejercicios Guiados

Cierre

Dos Problemas Frecuentes

Dos Problemas Frecuentes

¿Qué problema queremos resolver?

Dos situaciones distintas

1. **Resolver una ecuación:** encontrar los valores de x donde un polinomio vale cero.
2. **Ajustar datos:** encontrar una recta o curva polinómica que represente mediciones con ruido.

Herramientas de hoy

- `np.roots`: resuelve ecuaciones polinómicas.
- `np.polyfit`: ajusta un polinomio a datos.

Dos Problemas Frecuentes

Mapa mental: **roots** vs **polyfit**

np.roots

- Entrada: coeficientes del polinomio.
- Salida: raíces.
- Pregunta típica: ¿cuándo $p(x) = 0$?

np.polyfit

- Entrada: datos x, datos y, grado.
- Salida: coeficientes del ajuste.
- Pregunta típica: ¿qué modelo representa mis datos?

Idea clave

roots trabaja con una ecuación ya conocida. **polyfit** construye un modelo desde datos.

Resolver Polinomios con `np.roots`

Resolver Polinomios con `np.roots`

Polinomios como listas de coeficientes

Para usar `np.roots`, los coeficientes se escriben desde la potencia mayor hasta el término independiente.

Ejemplo

$$x^3 - 6x^2 + 11x - 6 = 0$$

$$\text{coef} = [1, -6, 11, -6]$$

- Si falta una potencia, se coloca coeficiente cero.
- $x^4 - 5x^2 + 4 = 0$ se escribe como $[1, 0, -5, 0, 4]$.

Resolver Polinomios con `np.roots`

Sintaxis básica de **`np.roots`**

```
1 import numpy as np
2
3 #  $x^3 - 6x^2 + 11x - 6 = 0$ 
4 coef = [1, -6, 11, -6]
5
6 raices = np.roots(coef)
7 print(raices)
```

Interpretación

La salida contiene todos los valores de x que hacen que el polinomio sea cero.

Resolver Polinomios con `np.roots`

Raíces reales y raíces complejas

`np.roots` puede devolver números complejos. En problemas físicos muchas veces interesa filtrar las raíces reales.

```
1 raices_reales = []
2
3 for raiz in raices:
4     if abs(raiz.imag) < 1e-10:
5         raices_reales.append(raiz.real)
6
7 print(raices_reales)
```

Criterio práctico

Usamos `abs(raiz.imag) < 1e-10` porque los cálculos numéricos pueden dejar partes imaginarias muy pequeñas.

Ajustar Datos con `np.polyfit`

Ajustar Datos con `np.polyfit`

Ajuste lineal

Si esperamos una relación aproximadamente lineal,

$$y = mx + b,$$

podemos estimar la pendiente m y el intercepto b desde datos experimentales.

Sintaxis

```
m, b = np.polyfit(x, y, deg=1)
```

- m : pendiente del ajuste.
- b : intercepto del ajuste.
- **deg=1**: polinomio de grado 1, es decir, recta.

Ajustar Datos con `np.polyfit`

Flujo mínimo de ajuste

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 m, b = np.polyfit(x, y, deg=1)
5 y_fit = m*x + b
6
7 plt.scatter(x, y, label="datos")
8 plt.plot(x, y_fit, "r-", label="ajuste")
9 plt.xlabel("x")
10 plt.ylabel("y")
11 plt.legend()
12 plt.grid(True, alpha=0.3)
13 plt.show()
```

Ajustar Datos con `np.polyfit`

Calidad del ajuste: residuos y R^2

Después de ajustar, conviene evaluar si la recta realmente representa los datos.

```
1 residuos = y - y_fit
2
3 SS_res = np.sum(residuos**2)
4 SS_tot = np.sum((y - np.mean(y))**2)
5 R2 = 1 - SS_res/SS_tot
6
7 print(f"R2 = {R2:.4f}")
```

Interpretación

R^2 cercano a 1 indica que el modelo explica gran parte de la variación de los datos.

Ejercicios Guiados



Enunciado

La altura de un proyectil está dada por

$$h(t) = -4.9t^2 + 18t + 1.5.$$

- Resolver $h(t) = 0$ usando `np.roots`.
- Filtrar raíces reales.
- Identificar cuál raíz tiene sentido físico.
- Interpretar el resultado como tiempo de impacto con el suelo.

Conceptos: coeficientes, raíces reales, descarte de raíces no físicas.

Ejercicios Guiados

Ejercicio Guiado 2:

Calibración de un sensor



Enunciado

Un sensor entrega voltaje V para distintas temperaturas T . Se miden:

$$T = [0, 10, 20, 30, 40, 50] ^\circ\text{C}$$

$$V = [0.52, 0.71, 0.93, 1.11, 1.32, 1.51] \text{ V}$$

- Ajustar $V = mT + b$ con `np.polyfit`.
- Graficar datos y recta ajustada.
- Calcular residuos y R^2 .
- Interpretar pendiente e intercepto.

Cierre

np.roots

- Se usa cuando ya tenemos una ecuación polinómica.
- El resultado son valores de la variable independiente.
- Requiere interpretar raíces reales, complejas y físicamente válidas.

np.polyfit

- Se usa cuando tenemos datos experimentales.
- El resultado son coeficientes de un modelo.
- Requiere evaluar ajuste con gráfica, residuos y R^2 .

- Practicaremos 7 problemas distintos.
- Algunos serán de raíces, otros de ajuste, y uno combinará ambas ideas.
- El foco será decidir qué herramienta usar antes de escribir código.

Pregunta guía

¿Tengo una ecuación para resolver, o tengo datos para ajustar?